React Cheatsheet (Exam Ready)

Create React App with Vite

```
npm create vite@latest my-app -- --template react
```

Components & Props

1. Basic Functional Component

```
const HelloWorld = () => {
  return <h1>Hello, World!</h1>;
};

export default HelloWorld;
```

2. Component with Props

```
const Greet = (props) => {
  return <h2>Hello, {props.name}!</h2>;
};

// Usage: <Greet name="Hanine" />
```

3. Destructured Props

```
const GreetUser = ({ name, age }) => {
  return <p>Hello {name}, you are {age} years old.</p>;
};

// Usage: <GreetUser name="Hanine" age={20} />
```

4. Props with Children

```jsx
const Card = ({ title, children }) => {
  return (
    <div className="card">
      <h3>{title}</h3>
      <div>{children}</div>
    </div>
  );
};

// Usage:
// <Card title="My Card">
//   <p>Content goes here</p>
// </Card>
```

🔄 State with useState

1. Simple State

```jsx
import { useState } from "react";

const Counter = () => {
  const [count, setCount] = useState(0);

  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={() => setCount(count + 1)}>+</button>
      <button onClick={() => setCount(count - 1)}>-</button>
    </div>
  );
};
```

2. State with Props

```jsx
const PersonalCounter = ({ initialCount, name }) => {
  const [count, setCount] = useState(initialCount);

  return (
    <div>
      <h3>{name}'s Counter</h3>
      <p>Count: {count}</p>
      <button onClick={() => setCount(count + 1)}>+</button>
    </div>
  );
};

// Usage: <PersonalCounter name="Hanine" initialCount={5} />
```

3. Multiple State Variables

```
const UserProfile = () => {
  const [name, setName] = useState("");
  const [age, setAge] = useState(0);
  const [isActive, setIsActive] = useState(false);

  return (
    <div>
      <input value={name} onChange={(e) => setName(e.target.value)} />
      <input value={age} onChange={(e) => setAge(e.target.value)} />
      <button onClick={() => setIsActive(!isActive)}>
        {isActive ? "Active" : "Inactive"}
      </button>
    </div>
  );
};
```

4. Object State

```
const UserForm = () => {
  const [user, setUser] = useState({ name: "", email: "", age: "" });

  const handleChange = (e) => {
    setUser({ ...user, [e.target.name]: e.target.value });
  };

  return (
    <form>
      <input name="name" value={user.name} onChange={handleChange} />
      <input name="email" value={user.email} onChange={handleChange} />
      <input name="age" value={user.age} onChange={handleChange} />
    </form>
  );
};
```

5. Array State

```
const TodoList = () => {
  const [todos, setTodos] = useState(["Task 1", "Task 2"]);
  const [input, setInput] = useState("");

  const addTodo = () => {
    setTodos([...todos, input]);
    setInput("");
  };

  const removeTodo = (index) => {
    setTodos(todos.filter((_, i) => i !== index));
  };

  return (
    <div>
      <input value={input} onChange={(e) => setInput(e.target.value)} />
      <button onClick={addTodo}>Add</button>
      <ul>
        {todos.map((todo, index) => (
          <li key={index}>
            {todo} <button onClick={() => removeTodo(index)}>X</button>
          </li>
        ))}
      </ul>
    </div>
  );
};
```

🎯 Events

1. Button Click

```
const ButtonClick = () => {
  const handleClick = () => {
    alert("Button clicked!");
  };

  return <button onClick={handleClick}>Click Me</button>;
};
```

2. Click with State

```
const ToggleMessage = () => {
  const [isVisible, setIsVisible] = useState(false);

  return (
    <div>
      <button onClick={() => setIsVisible(!isVisible)}>Toggle</button>
      {isVisible && <p>Hello! You can see me now.</p>}
    </div>
  );
};
```

3. Passing Event to Parent

```
const ChildButton = ({ onButtonClick }) => {
  return <button onClick={onButtonClick}>Click from Child</button>;
};

const Parent = () => {
  const handleClick = () => alert("Clicked from child!");

  return <ChildButton onButtonClick={handleClick} />;
};
```

4. Input Change Event

```
const NameInput = () => {
  const [name, setName] = useState("");

  return (
    <div>
      <input
        type="text"
        value={name}
        onChange={(e) => setName(e.target.value)}
        placeholder="Enter your name"
      />
      <p>Your name is: {name}</p>
    </div>
  );
};
```

📝 Forms

1. Simple Form

```
const SimpleForm = () => {
  const [email, setEmail] = useState("");

  const handleSubmit = (e) => {
    e.preventDefault(); // Prevent page reload
    alert(`Submitted: ${email}`);
  };

  return (
    <form onSubmit={handleSubmit}>
      <input
        type="email"
        value={email}
        onChange={(e) => setEmail(e.target.value)}
      />
      <button type="submit">Submit</button>
    </form>
  );
};
```

2. Multiple Inputs

```
const MultiInputForm = () => {
  const [formData, setFormData] = useState({ username: "", age: "" });

  const handleChange = (e) => {
    const { name, value } = e.target;
    setFormData({ ...formData, [name]: value });
  };

  const handleSubmit = (e) => {
    e.preventDefault();
    console.log(formData);
  };

  return (
    <form onSubmit={handleSubmit}>
      <input
        name="username"
        value={formData.username}
        onChange={handleChange}
      />
      <input name="age" value={formData.age} onChange={handleChange} />
      <button type="submit">Submit</button>
    </form>
  );
};
```

3. Textarea

```
const TextAreaForm = () => {
  const [message, setMessage] = useState("");

  return (
    <form>
      <textarea value={message} onChange={(e) => setMessage(e.target.value)} />
      <p>{message}</p>
    </form>
  );
};
```

4. Select/Dropdown

```
const SelectForm = () => {
  const [car, setCar] = useState("Volvo");

  return (
    <form>
      <select value={car} onChange={(e) => setCar(e.target.value)}>
        <option value="Ford">Ford</option>
        <option value="Volvo">Volvo</option>
        <option value="Fiat">Fiat</option>
      </select>
      <p>Selected: {car}</p>
    </form>
  );
};
```

5. Checkbox

```
const CheckboxForm = () => {
  const [isChecked, setIsChecked] = useState(false);

  return (
    <form>
      <label>
        <input
          type="checkbox"
          checked={isChecked}
          onChange={(e) => setIsChecked(e.target.checked)}
        />
        I agree
      </label>
      <p>{isChecked ? "Agreed" : "Not agreed"}</p>
    </form>
  );
};
```

6. Radio Buttons

```jsx
const RadioForm = () => {
  const [gender, setGender] = useState("male");

  return (
    <form>
      <label>
        <input
          type="radio"
          value="male"
          checked={gender === "male"}
          onChange={(e) => setGender(e.target.value)}
        />
        Male
      </label>
      <label>
        <input
          type="radio"
          value="female"
          checked={gender === "female"}
          onChange={(e) => setGender(e.target.value)}
        />
        Female
      </label>
      <p>Selected: {gender}</p>
    </form>
  );
};
```

7. Form Validation

```
const EmailForm = () => {
  const [email, setEmail] = useState("");
  const [error, setError] = useState("");

  const handleSubmit = (e) => {
    e.preventDefault();

    if (!email.includes("@")) {
      setError("Please enter a valid email");
      return;
    }

    setError("");
    alert("Form submitted successfully!");
  };

  return (
    <form onSubmit={handleSubmit}>
      <input
        type="text"
        value={email}
        onChange={(e) => setEmail(e.target.value)}
      />
      {error && <p style={{ color: "red" }}>{error}</p>}
      <button type="submit">Submit</button>
    </form>
  );
};
```

8. Complete Registration Form

```
const RegistrationForm = () => {
  const [formData, setFormData] = useState({
    username: "",
    email: "",
    password: "",
    country: "USA",
    agreeTerms: false,
  });

  const handleChange = (e) => {
    const { name, value, type, checked } = e.target;
    setFormData({
      ...formData,
      [name]: type === "checkbox" ? checked : value,
    });
  };

  const handleSubmit = (e) => {
    e.preventDefault();
    console.log("Form Data:", formData);
  };

  return (
    <form onSubmit={handleSubmit}>
      <input
        name="username"
        value={formData.username}
        onChange={handleChange}
        placeholder="Username"
      />
      <input
        type="email"
        name="email"
        value={formData.email}
        onChange={handleChange}
        placeholder="Email"
      />
      <input
        type="password"
        name="password"
        value={formData.password}
        onChange={handleChange}
        placeholder="Password"
      />
      <select name="country" value={formData.country} onChange={handleChange}>
        <option value="USA">USA</option>
        <option value="Canada">Canada</option>
        <option value="UK">UK</option>
      </select>
      <label>
        <input
          type="checkbox"
          name="agreeTerms"
          checked={formData.agreeTerms}
          onChange={handleChange}
```

```
        />
        I agree to terms
      </label>
      <button type="submit">Register</button>
    </form>
  );
};
```

✅ Conditional Rendering

1. If/Else with Ternary

```
const ShowHide = () => {
  const [isVisible, setIsVisible] = useState(true);

  return (
    <div>
      <button onClick={() => setIsVisible(!isVisible)}>Toggle</button>
      {isVisible ? <p>I am visible</p> : <p>I am hidden</p>}
    </div>
  );
};
```

2. AND Operator (&&)

```
const Notification = () => {
  const [hasMessage, setHasMessage] = useState(true);

  return <div>{hasMessage && <p>You have a new message!</p>}</div>;
};
```

3. Multiple Conditions

```
const UserStatus = ({ isLoggedIn, isPremium }) => {
  return (
    <div>
      {isLoggedIn ? (
        isPremium ? (
          <p>Welcome Premium User!</p>
        ) : (
          <p>Welcome User!</p>
        )
      ) : (
        <p>Please log in</p>
      )}
    </div>
  );
};
```

4. Switch-like Pattern

```
const StatusMessage = ({ status }) => {
  const messages = {
    loading: "Loading...",
    success: "Success!",
    error: "Error occurred",
  };

  return <p>{messages[status] || "Unknown status"}</p>;
};
```

📋 Lists & Arrays

1. Rendering Lists

```
const ItemList = ({ items }) => {
  return (
    <ul>
      {items.map((item, index) => (
        <li key={index}>{item}</li>
      ))}
    </ul>
  );
};

// Usage: <ItemList items={['Apple', 'Banana', 'Orange']} />
```

2. List with Objects

```
const UserList = () => {
  const users = [
    { id: 1, name: "Alice", age: 25 },
    { id: 2, name: "Bob", age: 30 },
    { id: 3, name: "Charlie", age: 35 },
  ];

  return (
    <ul>
      {users.map((user) => (
        <li key={user.id}>
          {user.name} - {user.age} years old
        </li>
      ))}
    </ul>
  );
};
```

3. Filter Array

```
const FilteredList = () => {
  const [items] = useState(["Apple", "Banana", "Orange", "Grape"]);
  const [filter, setFilter] = useState("");

  const filteredItems = items.filter((item) =>
    item.toLowerCase().includes(filter.toLowerCase())
  );

  return (
    <div>
      <input
        type="text"
        placeholder="Search..."
        value={filter}
        onChange={(e) => setFilter(e.target.value)}
      />
      <ul>
        {filteredItems.map((item, index) => (
          <li key={index}>{item}</li>
        ))}
      </ul>
    </div>
  );
};
```

4. Map Array

```
const PriceList = () => {
  const prices = [10, 20, 30, 40];
  const discountedPrices = prices.map((price) => price * 0.9);

  return (
    <ul>
      {discountedPrices.map((price, index) => (
        <li key={index}>${price.toFixed(2)}</li>
      ))}
    </ul>
  );
};
```

5. Find in Array

```
const FindUser = () => {
  const users = [
    { id: 1, name: "Alice" },
    { id: 2, name: "Bob" },
  ];

  const user = users.find((u) => u.id === 2);

  return <p>Found: {user ? user.name : "Not found"}</p>;
};
```

6. Some & Every

```
const CheckAges = () => {
  const ages = [18, 20, 25, 30];

  const hasAdult = ages.some((age) => age >= 18); // true
  const allAdults = ages.every((age) => age >= 18); // true

  return (
    <div>
      <p>Has adult: {hasAdult ? "Yes" : "No"}</p>
      <p>All adults: {allAdults ? "Yes" : "No"}</p>
    </div>
  );
};
```

7. Reduce Array

```
const TotalPrice = () => {
  const prices = [10, 20, 30, 40];
  const total = prices.reduce((sum, price) => sum + price, 0);

  return <p>Total: ${total}</p>;
};
```

8. Sort Array

```
const SortedList = () => {
  const [items, setItems] = useState([
    { id: 3, name: "Charlie" },
    { id: 1, name: "Alice" },
    { id: 2, name: "Bob" },
  ]);

  const sortById = () => {
    const sorted = [...items].sort((a, b) => a.id - b.id);
    setItems(sorted);
  };

  const sortByName = () => {
    const sorted = [...items].sort((a, b) => a.name.localeCompare(b.name));
    setItems(sorted);
  };

  return (
    <div>
      <button onClick={sortById}>Sort by ID</button>
      <button onClick={sortByName}>Sort by Name</button>
      <ul>
        {items.map((item) => (
          <li key={item.id}>
            {item.id} - {item.name}
          </li>
        ))}
      </ul>
    </div>
  );
};
```

🌐 Fetch API (Data Fetching)

1. Fetch on Button Click

```
const FetchOnClick = () => {
  const [data, setData] = useState(null);
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState(null);

  const fetchData = () => {
    setLoading(true);
    setError(null);

    fetch("https://api.example.com/data")
      .then((response) => {
        if (!response.ok) throw new Error("Network response was not ok");
        return response.json();
      })
      .then((data) => {
        setData(data);
        setLoading(false);
      })
      .catch((error) => {
        setError(error.message);
        setLoading(false);
      });
  };

  return (
    <div>
      <button onClick={fetchData}>Fetch Data</button>
      {loading && <p>Loading...</p>}
      {error && <p>Error: {error}</p>}
      {data && <pre>{JSON.stringify(data, null, 2)}</pre>}
    </div>
  );
};
```

2. Fetch with async/await

```
const FetchAsync = () => {
  const [users, setUsers] = useState([]);
  const [loading, setLoading] = useState(false);

  const fetchUsers = async () => {
    setLoading(true);
    try {
      const response = await fetch("https://jsonplaceholder.typicode.com/users");
      const data = await response.json();
      setUsers(data);
    } catch (error) {
      console.error("Error:", error);
    } finally {
      setLoading(false);
    }
  };

  return (
    <div>
      <button onClick={fetchUsers}>Load Users</button>
      {loading && <p>Loading...</p>}
      <ul>
        {users.map((user) => (
          <li key={user.id}>{user.name}</li>
        ))}
      </ul>
    </div>
  );
};
```

🎨 Styling in React

1. Inline Styles

```
const StyledComponent = () => {
  return (
    <div style={{ color: "blue", fontSize: "20px", marginTop: "10px" }}>
      Styled Text
    </div>
  );
};

// With variables
const MyComponent = () => {
  const divStyle = {
    color: "red",
    backgroundColor: "lightgray",
    padding: "10px",
  };

  return <div style={divStyle}>Content</div>;
};
```

2. CSS Classes (className)

```
// Component
const Card = () => {
  return <div className="card primary">Card Content</div>;
};

// Conditional classes
const Button = ({ isPrimary }) => {
  return (
    <button className={isPrimary ? "btn btn-primary" : "btn btn-secondary"}>
      Click Me
    </button>
  );
};

// Multiple conditional classes
const Alert = ({ type, isVisible }) => {
  const classes = `alert ${type} ${isVisible ? "show" : "hide"}`;
  return <div className={classes}>Alert Message</div>;
};
```

3. Dynamic Styles

```
const DynamicButton = () => {
  const [isActive, setIsActive] = useState(false);

  return (
    <button
      style={{
        backgroundColor: isActive ? "green" : "gray",
        color: isActive ? "white" : "black",
      }}
      onClick={() => setIsActive(!isActive)}
    >
      {isActive ? "Active" : "Inactive"}
    </button>
  );
};
```

4. CSS Modules (if used in project)

```
// Import CSS module
import styles from "./MyComponent.module.css";

const MyComponent = () => {
  return (
    <div className={styles.container}>
      <h1 className={styles.title}>Title</h1>
    </div>
  );
};
```

💡 Bonus: Updating State with Spread Operator

Updating Arrays

```
// Add item to end
setItems([...items, newItem]);

// Add item to beginning
setItems([newItem, ...items]);

// Remove item by index
setItems(items.filter((_, index) => index !== indexToRemove));

// Update item at specific index
setItems(items.map((item, index) =>
  index === targetIndex ? updatedItem : item
));

// Update object in array by id
setUsers(users.map(user =>
  user.id === targetId ? { ...user, name: "New Name" } : user
));

// Replace entire array
setItems([...newArray]);
```

Updating Objects

```
// Update single property
setUser({ ...user, name: "New Name" });

// Update multiple properties
setUser({ ...user, name: "John", age: 30 });

// Update nested object
setUser({ ...user, address: { ...user.address, city: "Paris" } });

// Update using computed property name
const field = "email";
setUser({ ...user, [field]: "new@email.com" });

// Add new property
setUser({ ...user, newProperty: "value" });

// Toggle boolean property
setUser({ ...user, isActive: !user.isActive });
```

🎨 Quick Tips

JSX Rules

- Use `className` instead of `class`
- Use `htmlFor` instead of `for`
- Self-closing tags need `/` (e.g., `<img />`)
- JavaScript in JSX: use `{}`
- Inline styles: `style={{ color: "red", fontSize: "20px" }}`

State Best Practices

- Never mutate state directly: ❌ `state.push(item)` ✅ `setState([...state, item])`
- For objects: spread and override: `setState({ ...state, name: "new" })`
- For arrays: use spread, filter, map to create new arrays

Common Patterns

```
// Toggle boolean
setIsActive(!isActive);

// Toggle with callback (safer)
setIsActive((prev) => !prev);

// Add to array
setItems([...items, newItem]);

// Remove from array by index
setItems(items.filter((_, i) => i !== index));

// Update object in array
setUsers(users.map((u) => (u.id === id ? { ...u, name: "new" } : u)));
```

Array Methods Summary

```
// filter - returns new array with items that pass test
const adults = users.filter((user) => user.age >= 18);

// map - transforms each item
const names = users.map((user) => user.name);

// find - returns first item that matches
const user = users.find((user) => user.id === 1);

// some - returns true if ANY item passes test
const hasAdmin = users.some((user) => user.role === "admin");

// every - returns true if ALL items pass test
const allActive = users.every((user) => user.isActive);

// reduce - reduces array to single value
const total = prices.reduce((sum, price) => sum + price, 0);

// sort - sorts array (mutates, so use spread first)
const sorted = [...items].sort((a, b) => a - b);
```

🚀 Common Exam Patterns

Todo App Pattern

```
const TodoApp = () => {
  const [todos, setTodos] = useState([]);
  const [input, setInput] = useState("");

  const addTodo = () => {
    setTodos([...todos, { id: Date.now(), text: input, done: false }]);
    setInput("");
  };

  const toggleTodo = (id) => {
    setTodos(todos.map((t) => (t.id === id ? { ...t, done: !t.done } : t)));
  };

  const deleteTodo = (id) => {
    setTodos(todos.filter((t) => t.id !== id));
  };

  return (
    <div>
      <input value={input} onChange={(e) => setInput(e.target.value)} />
      <button onClick={addTodo}>Add</button>
      <ul>
        {todos.map((todo) => (
          <li key={todo.id}>
            <span
              style={{ textDecoration: todo.done ? "line-through" : "none" }}
              onClick={() => toggleTodo(todo.id)}
            >
              {todo.text}
            </span>
            <button onClick={() => deleteTodo(todo.id)}>X</button>
          </li>
        ))}
      </ul>
    </div>
  );
};
```

Counter with Min/Max

```
const LimitedCounter = () => {
  const [count, setCount] = useState(0);
  const MIN = 0;
  const MAX = 10;

  const increment = () => {
    if (count < MAX) setCount(count + 1);
  };

  const decrement = () => {
    if (count > MIN) setCount(count - 1);
  };

  return (
    <div>
      <button onClick={decrement} disabled={count <= MIN}>
        -
      </button>
      <span>{count}</span>
      <button onClick={increment} disabled={count >= MAX}>
        +
      </button>
    </div>
  );
};
```

Form with Validation Pattern

```
const ValidatedForm = () => {
  const [form, setForm] = useState({ email: "", password: "" });
  const [errors, setErrors] = useState({});

  const validate = () => {
    const newErrors = {};
    if (!form.email.includes("@")) newErrors.email = "Invalid email";
    if (form.password.length < 6) newErrors.password = "Too short";
    setErrors(newErrors);
    return Object.keys(newErrors).length === 0;
  };

  const handleSubmit = (e) => {
    e.preventDefault();
    if (validate()) {
      console.log("Form valid:", form);
    }
  };

  return (
    <form onSubmit={handleSubmit}>
      <input
        name="email"
        value={form.email}
        onChange={(e) => setForm({ ...form, email: e.target.value })}
      />
      {errors.email && <span>{errors.email}</span>}

      <input
        name="password"
        type="password"
        value={form.password}
        onChange={(e) => setForm({ ...form, password: e.target.value })}
      />
      {errors.password && <span>{errors.password}</span>}

      <button type="submit">Submit</button>
    </form>
  );
};
```